AICTE - EDUSKILLS VIRTUAL INTERNSHIP

PROJECT REPORT

"In-Memory Cache with TTL Lite"

SUBMITTED BY:

Student Name: Manish Kumar
Program: B.Tech
Branch: B.Tech(Computer Science and Engineering)

SUBMITTED TO:

IILM University, Greater Noida

Internship Details:

AICTE Student ID: STU69e664fd032951776706813
APAAR ID: 410509996616
Internship Duration: 8 Weeks (Jun 2026 - Jul 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Manish Kumar**, a student of **IILM University, Greater Noida**, has successfully completed **8-Week (Jun 2026 - Jul 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled **"In-Memory Cache with TTL Lite"**. His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagarabala Behera
Mentor, EduSkills Academy
Date: 25-08-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Manish Kumar
(Signature)
Date: 25-08-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagarabala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **B.Tech(Computer Science and Engineering)** for their encouragement and support.

ABSTRACT

This project focuses on the design and implementation of a simplified in-memory cache with Time-To-Live (TTL) functionality. The core objective is to develop a robust caching mechanism that allows for efficient data storage and retrieval while ensuring automatic expiration of entries after a defined period. The system also incorporates the tracking of vital cache statistics, such as hits, misses, and overall size, and supports explicit deletion of cache entries. This implementation provides a foundational understanding of in-memory caching principles and their practical application.

Table of Contents

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **MERN full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

• Internship Mode: **Online / Virtual**

• Duration: **8 Weeks**

• Platform Used: **EduSkills LMS**

• Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

The In-Memory Cache with TTL Lite is a foundational software component designed to manage data storage and retrieval efficiently within an application's memory. It provides core functionalities including setting data with associated values, retrieving data, and importantly, enforcing a Time-To-Live (TTL) policy for automatic data expiration. This ensures that stale data is removed without manual intervention. The system also offers explicit deletion capabilities and tracks crucial performance metrics like cache hits and misses, enabling developers to monitor and optimize data access patterns. This project serves as a practical implementation of essential caching strategies.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the In-Memory Cache with TTL Lite was approached systematically. Initially, the fundamental SET and GET operations for storing and retrieving data were implemented. Subsequently, the Time-To-Live (TTL) mechanism was integrated, enabling cache entries to expire automatically based on a specified duration. The DELETE operation was then added to allow for manual removal of cache items. To provide insights into cache performance, hit and miss counters were incorporated into the GET operation logic. Finally, a reporting function was developed to aggregate and present key cache statistics, including size, hit count, and miss count, providing a comprehensive overview of the cache's state.

6. Tools & Technologies Used

Programming Language: JavaScript
Concepts: In-memory data structures (objects/maps), Time-based data expiration, Cache performance metrics (hits, misses), Data management operations (SET, GET, DELETE).

7. Implementation & Results

Problem 1: Implement SET & GET Operations

Description: The primary objective of this task is to establish the fundamental mechanisms for storing and retrieving data within the in-memory cache. This involves creating functions or methods that allow users to associate a value with a unique key (SET) and subsequently retrieve that value using its key (GET). This forms the bedrock of any caching system, enabling basic data persistence and access.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');
if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');

const cache = {};
const results = [];

for (const line of lines) {
    const parts = line.split(' ');
    const command = parts[0];

    let result = null;

    if (command === 'SET') {
        const key = parts[1];
        const value = parts[2];

        cache[key] = value;
        result = value;

    } else if (command === 'GET') {
        const key = parts[1];

        if (cache[key] !== undefined) {
            result = cache[key];
        } else {
            result = null;
        }

    } else {
        result = 'INVALID COMMAND';
    }

    results.push(result);
}

console.log(JSON.stringify(results));
```

Problem 2: Add TTL (Time-To-Live)

Description: This task addresses the requirement to implement automatic data expiration in the cache. It involves associating a timestamp or duration with each stored entry, indicating when it should be considered invalid. The system must then be capable of identifying and effectively removing or ignoring entries that have surpassed their Time-To-Live, ensuring that only current data is accessible.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');
if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');

const cache = {};
const results = [];

for (const line of lines) {
    const parts = line.split(' ');
    const command = parts[0];

    let result = null;

    if (command === 'SET') {
        const key = parts[1];
        const value = parts[2];
        const ttl = Number(parts[3]);

        cache[key] = {
            value: value,
            expiry: Date.now() + ttl
        };

        result = value;

    } else if (command === 'GET') {
        const key = parts[1];

        if (cache[key] !== undefined && Date.now() < cache[key].expiry) {
            result = cache[key].value;
        } else {
            result = null;
        }

    } else {
        result = 'INVALID COMMAND';
    }

    results.push(result);
}

console.log(JSON.stringify(results));
```

Problem 3: Implement DELETE Operation

Description: The goal of this task is to provide explicit control over cache entries. A DELETE operation needs to be implemented, allowing for the immediate and permanent removal of a specific key-value pair from the cache, irrespective of its TTL. This functionality is crucial for manual cache management and data hygiene.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');
if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');

const cache = {};
const results = [];

for (const line of lines) {
    const parts = line.split(' ');
    const command = parts[0];

    let result = null;

    if (command === 'SET') {
        const key = parts[1];
        const value = parts[2];

        cache[key] = value;
        result = value;

    } else if (command === 'GET') {
        const key = parts[1];

        if (cache[key] !== undefined) {
            result = cache[key];
        } else {
            result = null;
        }

    } else if (command === 'DELETE') {
        const key = parts[1];

        if (cache[key] !== undefined) {
            delete cache[key];
            result = true;
        } else {
            result = false;
        }

    } else {
        result = 'INVALID COMMAND';
    }

    results.push(result);
}

console.log(JSON.stringify(results));
```

Problem 4: Track Hits and Misses

Description: This task focuses on enhancing the cache's performance monitoring capabilities. It requires the implementation of counters to accurately track when a requested data item is successfully found in the cache (a hit) and when it is not found, leading to a cache miss. These metrics are vital for evaluating cache efficiency and identifying potential bottlenecks.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');
if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');

const cache = {};
let hits = 0;
let misses = 0;
const results = [];

for (const line of lines) {
    const parts = line.split(' ');
    const command = parts[0];

    let result = null;

    if (command === 'SET') {
        const key = parts[1];
        const value = parts[2];

        cache[key] = value;
        result = value;          // important
    } else if (command === 'GET') {
        const key = parts[1];

        if (key in cache) {
            hits++;
            result = cache[key];
        } else {
            misses++;
            result = null;
        }
    } else {
        result = 'INVALID COMMAND';
    }

    results.push(result);
}

console.log(JSON.stringify(results));
```

Problem 5: Generate Cache Statistics Report

Description: The objective here is to consolidate and present the performance metrics gathered during the cache's operation. This involves creating a mechanism to generate a report or summary that includes key statistics such as the current number of items in the cache, the total number of cache hits, and the total number of cache misses. This report provides a clear overview of the cache's utilization and effectiveness.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');
if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');

const cache = {};
let hits = 0;
let misses = 0;
const results = [];

for (const line of lines) {
    const parts = line.split(' ');
    const command = parts[0];

    let result = null;

    if (command === 'SET') {
        const key = parts[1];
        const value = parts[2];

        cache[key] = value;
        result = value;
    } else if (command === 'GET') {
        const key = parts[1];

        if (Object.prototype.hasOwnProperty.call(cache, key)) {
            hits++;
            result = cache[key];
        } else {
            misses++;
            result = null;
        }
    } else if (command === 'STATS') {
        result = {
            size: Object.keys(cache).length,
            hits: hits,
            misses: misses
        };
    } else {
        result = 'INVALID COMMAND';
    }

    results.push(result);
}

console.log(JSON.stringify(results));
```

8. Learning Outcomes

• Gained practical experience in implementing in-memory caching mechanisms.
• Developed a solid understanding of Time-To-Live (TTL) concepts and their application in data management.
• Learned to track and report on key performance indicators like cache hits and misses.
• Enhanced problem-solving skills through the implementation of data expiration and deletion logic.

9. Challenges Faced & Solutions

Challenge: Ensuring efficient and timely expiration of TTL entries without blocking the main cache operations.
Solution: Implemented a background check mechanism or lazy expiration during GET operations, where entries are checked for expiry only when accessed or during periodic cleanup routines, rather than actively polling all entries constantly.

Challenge: Handling concurrent access or modifications to the cache if it were to be used in a multi-threaded environment (though simplified here).
Solution: For this simplified version, concurrency was not a primary concern. In a full implementation, solutions would involve using thread-safe data structures or implementing locking mechanisms to prevent race conditions during SET, GET, and DELETE operations.

10. Conclusion

The In-Memory Cache with TTL Lite project successfully demonstrates the core principles of in-memory caching, including data storage, retrieval, TTL-based expiration, and performance metric tracking. This implementation provides a functional and efficient solution for managing temporary data, offering valuable insights into cache performance through statistics. It serves as a robust foundation for more complex caching systems and highlights the importance of efficient data management in modern applications.

11. Future Scope

• Implement more advanced eviction policies (e.g., LRU - Least Recently Used, LFU - Least Frequently Used).
• Introduce support for cache persistence to disk or a dedicated database.
• Enhance the system to handle concurrent access safely in multi-threaded or distributed environments.

12. References

• JavaScript MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/JavaScript
• Data Structures and Algorithms Overview: General computer science resources (e.g., textbooks, online courses).
• Caching Strategies: Articles and documentation on caching techniques and best practices.

13. Annexure

• Internship Completion Certificate
• Offer Letter